Week 7 – 2D Lists / Arrays

■ Learning Objectives

• Understand how to represent tables or grids using 2D lists.
• Access, modify, and traverse elements with nested loops.
• Apply 2D list processing to solve simulation and math problems.

■ Key Concepts

1. Creating 2D Lists: Use a list of lists, e.g. grid = [[0,1,2],[3,4,5]].
2. Access Elements: Use two indices grid[r][c].
3. Traversing: Nested loops for rows and columns.
4. Applications: Tables, maps, games, matrix operations.

Example 1 – Print All Elements of a 2D List

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for r in range(len(grid)):
    for c in range(len(grid[r])):
        print(grid[r][c], end=" ")
    print()
```

Example 2 – Row and Column Sums

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Sum of each row
for row in grid:
    print("Row sum:", sum(row))

# Sum of each column
cols = len(grid[0])
for c in range(cols):
    total = 0
    for r in range(len(grid)):
        total += grid[r][c]
    print("Col", c, "sum:", total)
```

■ Practice Problems

Practice 1 – Transpose a Matrix

Given a 2D list, create a new list that is its transpose (rows ↔ columns).

Solution:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]

rows, cols = len(matrix), len(matrix[0])
transpose = []
for c in range(cols):
    row = []
    for r in range(rows):
        row.append(matrix[r][c])
    transpose.append(row)

print(transpose)  # [[1,4],[2,5],[3,6]]
```

Practice 2 – Find Maximum in 2D List

Find the largest element and its (row, column) position.

Solution:

```
grid = [
    [5, 7, 3],
    [9, 2, 4],
    [6, 8, 1]
]

max_val = grid[0][0]
pos = (0, 0)
for r in range(len(grid)):
    for c in range(len(grid[r])):
        if grid[r][c] > max_val:
            max_val = grid[r][c]
            pos = (r, c)

print("Max:", max_val, "at", pos)
```

Practice 3 – Check if Matrix is Symmetric

A matrix is symmetric if M[i][j] == M[j][i] for all i, j.

Solution:

```
M = [
    [1, 2, 3],
    [2, 4, 5],
    [3, 5, 6]
]

symmetric = True
for i in range(len(M)):
    for j in range(len(M)):
        if M[i][j] != M[j][i]:
            symmetric = False
print("Symmetric:", symmetric)
```

■ Simulation Example – Counting Neighbors

Count how many '#' neighbors around each cell in a grid (used in game simulations).

```
grid = [
    ['#', '.', '#'],
    ['.', '#', '.'],
    ['#', '.', '.']
]

rows = len(grid)
cols = len(grid[0])
dirs = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]

for r in range(rows):
    for c in range(cols):
        count = 0
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '#':
                count += 1
        print(count, end=' ')
    print()
```

■ Homework Challenge

Write a program that reads a 2D grid of integers and prints the sum of each diagonal.

Expected Solution:

```
n = int(input())
grid = [list(map(int, input().split())) for _ in range(n)]

main_sum = 0
anti_sum = 0
for i in range(n):
    main_sum += grid[i][i]
    anti_sum += grid[i][n - i - 1]

print("Main diagonal sum:", main_sum)
print("Anti-diagonal sum:", anti_sum)
```